Module 12: Model Context Protocol (MCP)

GTK Cyber

Why MCP?

Where We Left Off

- In Module 11 we will built tools by hand-writing functions with the @tool decorator and binding them to a single LLM client.
- Every tool lived inside our own application code.
- What happens when we want those same tools across many apps, or want to use tools that other people built?

The M x N Problem

- AI applications need to connect to many external systems: files, databases, APIs, ticketing systems.
- Without a standard, each application builds custom glue for each system.
- M applications times N systems means M x N integrations to build and maintain.

The MCP Idea

- MCP defines one standard protocol for connecting AI applications to external systems.
- Build a system once as an MCP server and any MCP-aware application can use it.
- M x N custom integrations becomes M + N.
- Think of it as a USB-C port for AI applications.

What is MCP?

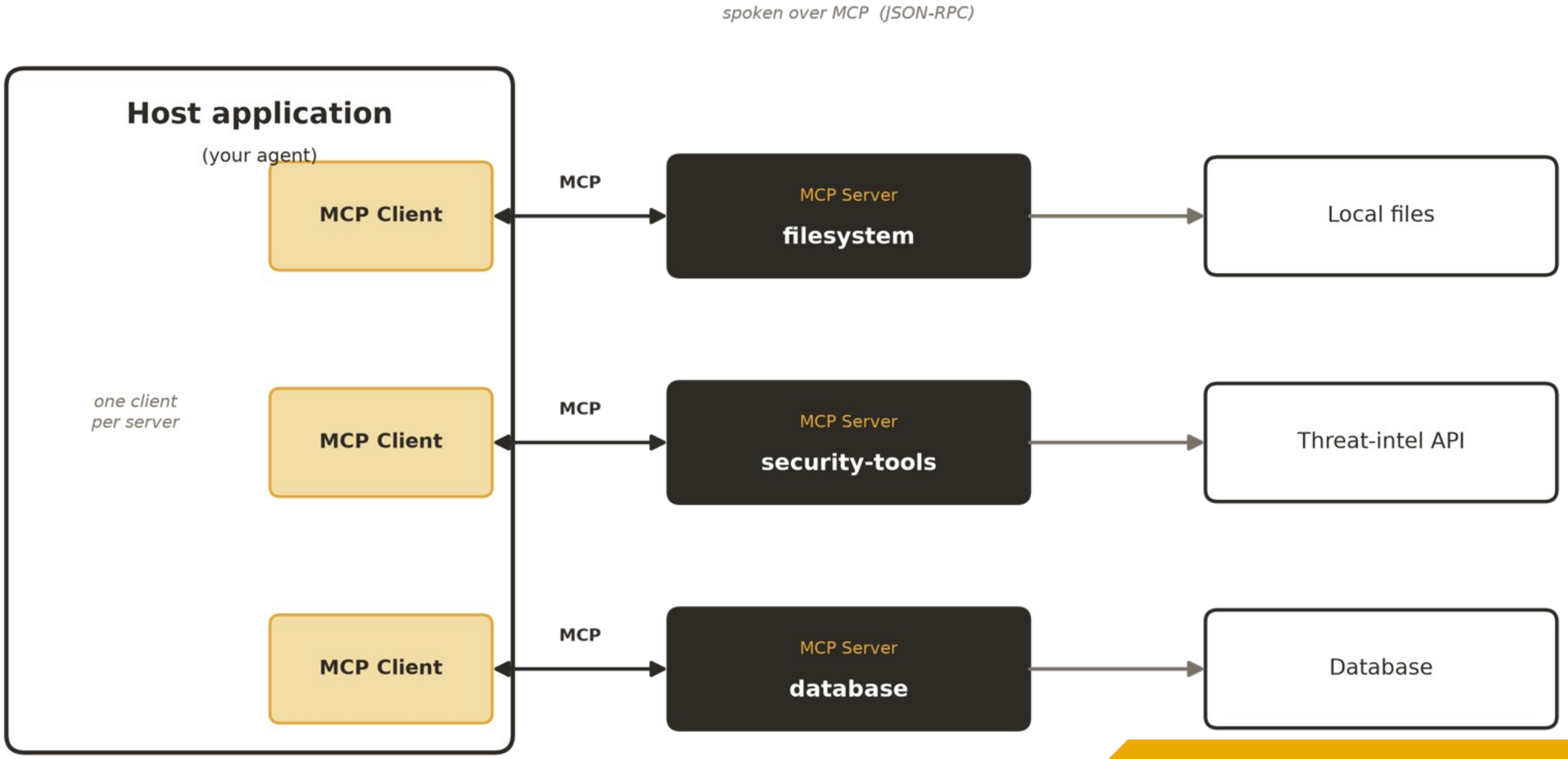
- An open protocol introduced by Anthropic in late 2024 and now broadly adopted across the industry.
- Built on JSON-RPC 2.0 messages.
- Standardizes how applications provide context and capabilities to LLMs.
- Language agnostic, with official SDKs for Python, TypeScript, and more.

MCP Architecture

Three Roles

- Host: the AI application the user interacts with, such as Claude Desktop, an IDE, or your agent.
- Client: lives inside the host and maintains a one-to-one connection to a server.
- Server: a program that exposes tools, data, and prompts over MCP.
- One host can run many clients, each talking to one server.

How the Pieces Fit



How They Connect

- The host creates one client for every server it wants to use.
- Each client speaks MCP to exactly one server.
- Servers can run locally on the same machine or remotely over the network.
- Servers expose capabilities and the host decides how to use them.

Transports

- MCP messages travel over a transport layer.
- stdio: the server runs as a local subprocess and communicates over standard input and output. Best for local tools.
- Streamable HTTP: the server runs remotely and communicates over HTTP. Best for shared or hosted servers.
- The protocol is identical regardless of transport.

Connection Lifecycle

- Initialize: client and server exchange protocol versions.
- Capability negotiation: each side declares what it supports.
- Operate: the client lists and calls tools, reads resources, and fetches prompts.
- Shutdown: the connection closes cleanly.

What a Server Exposes

Three Primitives

- An MCP server can expose three kinds of capability.
- Tools: actions the model can invoke.
- Resources: data the application can read for context.
- Prompts: reusable templates the user can invoke.
- The key difference is who is in control of each one.

Tools (Model-Controlled)

- Functions the LLM can choose to call, just like the tools from Module 11.
- Each tool has a name, a description, and an input schema.
- The model reads the description to decide when to call the tool.
- Examples: query a database, check a URL, send a message.

Resources (Application-Controlled)

- Read-only data the host can pull in for context.
- Addressed by a URI, for example file:///logs/today.log or db://users/42.
- The application decides when to load a resource, so the model does not pull it on its own.
- Examples: file contents, database records, configuration.

Prompts (User-Controlled)

- Reusable prompt templates the server offers to the user.
- Often surfaced as slash commands or menu items in the host.
- The user chooses when to invoke them.
- Examples: summarize this incident, draft a phishing report.

Client Capabilities (Briefly)

- Servers can also ask the client for things.
- Sampling: the server asks the host's LLM to generate text for it.
- Roots: the host tells the server which files or folders it may access.
- Elicitation: the server asks the user for more input.
- These let a server stay flexible without embedding its own model.

Building an MCP Server

Lab

Please complete MCP Lab: https://ai.gtkcyber.com/mcp-lab

The Python SDK

- The official MCP package provides FastMCP, a high-level way to build servers.
- Decorators turn plain Python functions into MCP tools, resources, and prompts.
- FastMCP handles the JSON-RPC protocol for you.
- You focus on the logic.

Define a Server and a Tool

```
from
mcp.server.fastmcp
import FastMCP
mcp =
FastMCP("security-
tools")
@mcp.tool()
def
check_url_reputation(
url: str) -> str:
   """Look up a URL
and return a risk
summary."""
   # your logic here
   return f"{url}:
malicious (score
87/100)"
```

Expose a Resource

```
@mcp.resource("io
c://blocklist")
def
get_blocklist() -
> str:
    """Return the
current IOC
blocklist."""
    return "
".join(load_known
_bad_domains())
```

Offer a Prompt

```
@mcp.prompt()
def phishing_report(sender:
str) -> str:
    """Template for a phishing
incident report."""
    return f"Draft an incident
report for a phishing email
from {sender}."
```

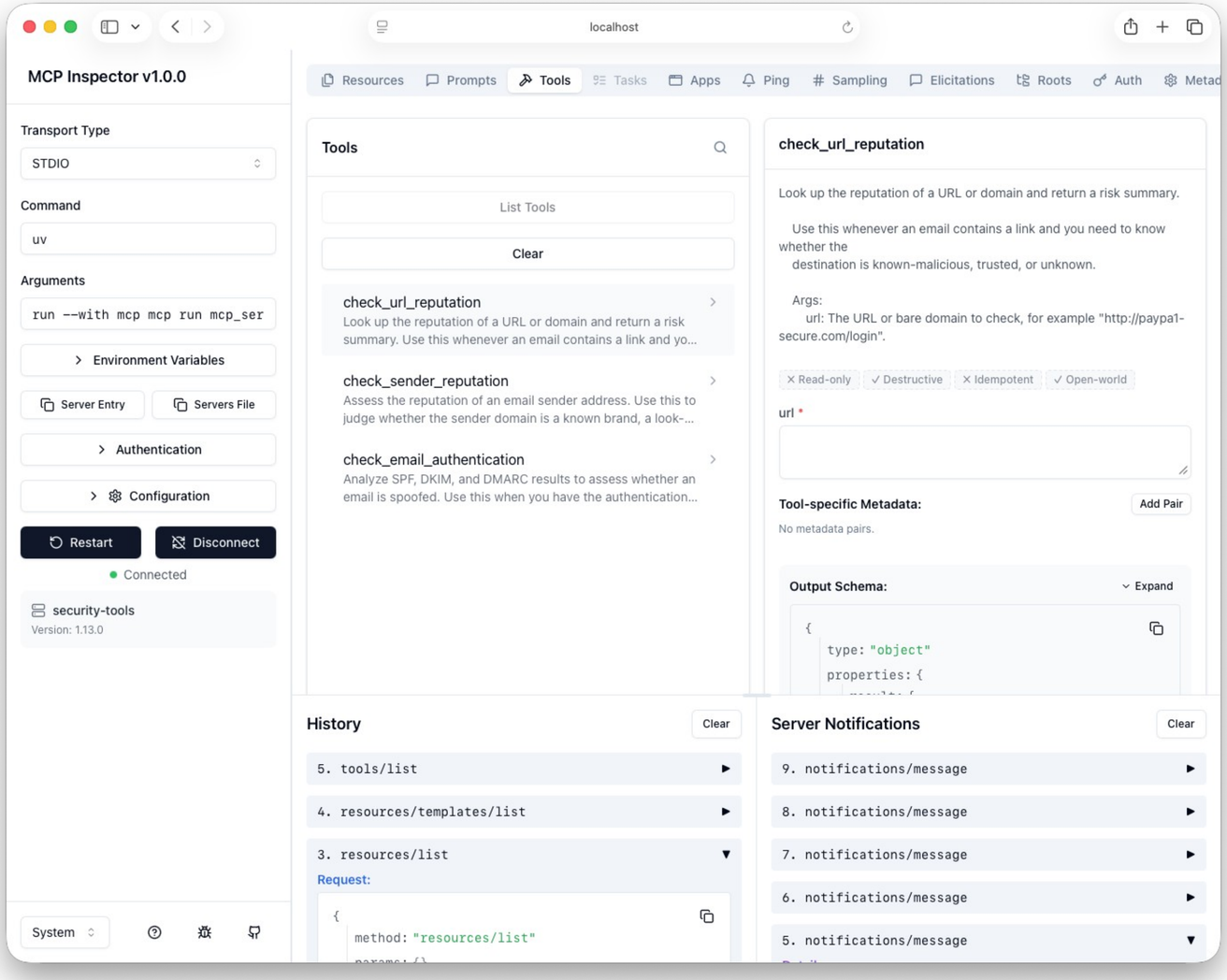
Run the Server

```
if __name__ ==
"__main__":
   mcp.run()
  # stdio transport
by default
```

The MCP Inspector

- A browser-based tool for testing a server without an agent.
- Lists the server's tools, resources, and prompts.
- Lets you call each one by hand and see the raw responses.
- Launch it with the `mcp dev` command pointed at your server file.
- Always inspect a server before wiring it to an agent.

MCP Inspector



Lab

Try out the MCP Inspector on the server `notebooks/mcp_server.py`.

```
mcp dev <filename>
```

Connecting a Server

How Hosts Connect

- A host is configured with a list of servers to launch.
- Configuration is usually a small JSON block naming the command to run.
- The host starts each server and opens a client to it.
- Claude Desktop, IDEs, and custom agents all follow this same pattern.

Server Configuration

```
{

"mcpServers"
: {

"security-
tools": {

"command":
"python",
     "args":
["mcp_server
.py"]
   }
 }
}
```

Using MCP From Your Agent

- The langchain-mcp-adapters package loads an MCP server's tools as LangChain tools.
- You hand those tools to create_agent exactly like before.
- Your agent no longer cares whether a tool is local code or a remote MCP server.

Load MCP Tools Into an Agent

```
from langchain_mcp_adapters.client
import MultiServerMCPClient
from langchain.agents import
create_agent
from langchain_anthropic import
ChatAnthropic
client = MultiServerMCPClient({
   "security-tools": {
       "command": "python",
       "args": ["mcp_server.py"],
       "transport": "stdio",
   }
})
tools = await client.get_tools()
llm = ChatAnthropic(model="claude-opus-
4-8")
agent = create_agent(llm, tools)
```

MCP and Security

A Bigger Attack Surface

- MCP makes it easy to plug external code into your agent.
- That convenience is also a risk.
- Every server you connect is code you are trusting with your agent's context and actions.
- A malicious or compromised server can do real damage. Recall the Replit example from Module 11.

Key Risks

- Untrusted servers: installing a server means running someone else's code.
- Tool poisoning: a tool description crafted to manipulate the model.
- Prompt injection: malicious text in a tool's output that steers the agent.
- Over-permissioning: giving a server more access than it needs.

Basic Defenses

- Vet and pin the servers you use, and prefer official or open-source ones.
- Give each server the least access it needs to do its job.
- Keep a human in the loop for destructive actions.
- Never expose secrets or credentials through a server you do not control.

The MCP Tax

- MCP is convenient, but that convenience has a running cost often called the "MCP tax".
- Every connected server's tool definitions (names, descriptions, and argument schemas) load into the model's context **on every request**.
- You pay for them in tokens and latency whether or not the model actually uses them.
- Connect many servers and the context fills with tool descriptions, raising cost and slowing every turn.
- Too many tools also degrade the model's choices, since it has more options to sift through.

Paying Less Tax

- Connect only the servers a task actually needs rather than everything at once.
- Expose a small set of high-value tools instead of dozens of narrow ones.
- Load or enable tools on demand where your framework supports it.
- Keep tool descriptions short, since they are re-sent on every request.
- Treat every tool you add as a recurring cost on each turn.

MCP or Hand-Rolled Tools?

- Hand-rolled tools from Module 11 are simplest when tools live in one app and you own them.
- MCP shines when you want to reuse tools across apps, share them, or use ones others built.
- Both give the model the same thing: callable tools.
- Start simple and reach for MCP when integration pressure appears.

The MCP Ecosystem

- A growing catalog of ready-made servers already exists.
- Filesystem, GitHub, Slack, databases, and many security tools.
- You can often connect an existing server instead of building one.
- Check the official servers repository before writing your own.

Bleeding Edge: AI Enabled Payments

Paying for It: Agent Payments

- As agents call more tools and APIs, they increasingly need to pay for them without a human in the loop.
- x402 (Coinbase, now Linux Foundation governed) revives the HTTP 402 "Payment Required" status: a server replies with a price, the agent attaches a signed stablecoin payment, and the server delivers once it settles onchain.
- MPP, the Machine Payments Protocol (Stripe and Tempo), meets the same need with a sessions model, where an agent pre-authorizes a limit once and then streams many micropayments. It is backwards-compatible with x402.
- Together they enable pay-per-call access to APIs, data, and compute in seconds, with no accounts or subscriptions.
- This pairs naturally with MCP, where a paid server or tool can charge per call.

Agent Payments: Security Implications

- Giving an agent a wallet turns a prompt injection into a way to spend real money, so treat payment as a high-risk action.
- x402 relies on off-chain "facilitators" to verify and settle payments, and the protocol does not enforce their correctness, so one faulty facilitator can affect many services.
- Researchers have demonstrated real attacks: replaying one payment for several grants, intercepting an authorization to charge without delivering, and steering agents to malicious endpoints with fake discovery listings.
- Basic defenses: hard spending limits, human approval for larger amounts, replay and idempotency protection, confirming settlement before delivery, and vetting who you pay.
- These standards are new and moving fast, so pilot carefully before trusting an agent to pay on its own.

Key Takeaways

- MCP is one standard protocol for connecting agents to tools, turning M x N custom integrations into M + N.
- A host runs one client per server, and each server exposes tools, resources, and prompts.
- Building a server is a few FastMCP decorators; connecting one to an agent is a few lines of code.
- MCP is not free: every connected server's tools cost context tokens and latency on every request.
- Every server you connect is code you trust, so vet servers, grant least privilege, and keep a human in the loop for risky actions.
- Agent payment protocols (x402, MPP) let agents pay per call, so only give an agent a wallet with strict limits.

Questions?

Lab

Please complete Lab 12.1: Building an MCP Server

The Lab: Build an MCP Server